LING 78100/73900
Assignment 5

Sorting and Searching

Due: November 8, 2026, end of day

1 Getting Started and Submission

Clone the `hw5` GitHub repository, finish the three search functions in the starter file, and send a pull request when you are done.

1. Clone the `hw5` GitHub repository to your computer. It contains the starter file `HW5-search.py` and the data file `./video_games.csv`.
2. Create a new branch for this assignment.
3. Complete the starter file. Keep the exact file name `HW5-search.py`.
4. Create a text file named `HW5-reflection.txt` with your answers to the Reflection section.
5. Commit as you go. When you are done, push your branch and open a pull request on GitHub that includes both `HW5-search.py` and `HW5-reflection.txt`.

The starter file runs as it is, but menu options 1 and 2 report "no matches" until you complete them.

2 Reminders Before You Start

Read the instructions carefully. If you have questions, ask me, but points are deducted for deviating from the assignment or missing questions.

- Keep the comment at the top of the file that describes the purpose of the program.
- Keep the `main()` function.
- Use informative variable names that describe the contents of the variables.
- Write your code incrementally and test as you go. Complete one function, run the program, then move on to the next.
- Passing the sample runs does not prove your program is correct. Come up with your own test cases and check the output.
- Have fun, start early, and ask questions when you have them.

3 The Dataset

The program explores a real-world dataset from CORGIS, the "Collection of Really Great, Interesting, Situated Datasets." It has records on over a thousand video games released between 2004 and 2008, including consoles, sales, review scores, and average playtime. We use a small subset of the features.

The data is stored in comma-separated value (CSV) format in `./video_games.csv`. The file is sorted by game title, ignoring upper and lower case, which is what makes binary search possible. Here are the first lines:

```
Title,MaxPlayers,Genre,ReviewScore,Sales,UsedPrice,Console,Rating,Year,PlayLength
007: From Russia with Love,1,Action / Racing / Driving,61,0.07,17.95,PSP,T,2006,6.38
300: March to Glory,1,Action,55,0.26,14.95,PSP,E,2007,7.4
7 Wonders of the Ancient World,1,Strategy,60,0.1,17.95,DS,T,2007,7.48
7 Wonders of the Ancient World,1,Strategy,51,0.08,17.95,PSP,E,2007,7.48
```

Each line has 10 features:

#	**Feature**	**Type**	**Description**
0	Title	string	Game title
1	MaxPlayers	integer	Maximum number of players
2	Genre	string	One or more genres, separated by slashes
3	ReviewScore	integer	Review score
4	Sales	float	Sales, measured in millions of dollars
5	UsedPrice	float	Average "used" price, in dollars
6	Console	string	Console the game was released on
7	Rating	string	Content rating
8	Year	integer	Release year
9	PlayLength	float	Average hours to play through the game (`-1` if not available)

4 What Is Already Provided

The starter file gives you a complete program except for the searches in options 1 and 2. It repeatedly shows this menu until the user chooses to quit:

```
1. Search by console and release year
2. Search by game title
3. Rank best reviewed games by console and genre
4. Find top selling games by year
5. Help: list possible Genres
6. Quit
```

These parts are written for you. Read them before you start, since you will call some of them from your own code:

- **Loading the data.** `load_data` reads the CSV and converts the numeric fields to `int` and `float`. Each game is a list, and named positions such as `TITLE`, `CONSOLE`, `YEAR`, and `REVIEW` let you write `game[YEAR]` instead of `game[8]`.
- **The menu and input checking.** The program rejects an invalid menu choice and any year outside 2004–2008.
- **Displaying results.** `display_results` takes a list of games and prints them as a table that fits in an 80-character terminal window, cutting long titles to 30 characters. Every search uses it.
- **Finding nearby matches.** `look_forward_backward` collects the matches next to a starting position. See the tips below.
- **Options 3, 4, and 5.** These rank games by review score or sales, and list the genres in alphabetical order. They use a provided `selection_sort`, return at most 25 results, and build the genre list from the data, so nothing is hardcoded. Genre searches match partially, so "Racing" also finds a game listed as "Action / Adventure / Racing / Driving".

You do not need to change any of this.

5 Your Task: Searching

Complete the three functions marked `TODO` in the "YOUR CODE" section of `HW5-search.py`. Keep the function names and parameters as they are, because the provided code calls them.

Function	Parameters	Returns
`linear_search_console_year`	`data, console, year`	A list of all games with that console and year, or an empty list
`binary_search_title`	`data, prefix`	The index of any game whose title starts with `prefix`, or `-1`
`search_by_title`	`data, prefix`	A list of all games whose title starts with `prefix`, in file order, or an empty list

Requirements:

- Search by console and year with linear search. Check every game and collect the matches in a list.
- Search by title with binary search. Do not loop through every title to find the first match.
- Compare in lowercase so the user's input is case insensitive, but return the games with their original capitalization.

- In `binary_search_title`, match with `startswith`, not `==`, so "LEGO" matches "LEGO Star Wars: The Complete Saga".
- In `search_by_title`, use your binary search result as the starting point and call `look_forward_backward` twice, once in each direction, to find the rest of the matches.
- You do not need to check that the console or title exists in the data. If nothing matches, return an empty list and the program reports that no matches were found.

Check your work with these counts from the data:

- Console `X360` and year `2005`: 18 games
- Title prefix `Sonic`: 12 games
- Title prefix `frogg`: 2 games
- Title prefix `LEGO`: 16 games

Also try your own cases, such as a title that sorts before the first game, one that sorts after the last, and a console typed in a different case.

6 Tips

6.1 Start with Option 1

Finish `linear_search_console_year` and test it from the menu before you begin the title search.

6.2 Adapting Linear Search

The version from class stops at the first match and returns its index (or `-1`). Yours must keep going, accumulate every match in a list, and return that list.

The data is a list of lists, so use double indexing, such as `game[CONSOLE]`, to reach a value inside each game.

6.3 Adapting Binary Search

Because the data is sorted by title, binary search can find a game quickly. But titles can be long, and the same game appears once per console it was released on. For example, there are 16 "LEGO" games across 5 consoles:

```
Legendary                          <- different prefix
LEGO Batman: The Videogame
LEGO Batman: The Videogame
...
LEGO Star Wars: The Complete Saga   <- suppose binary search lands here
```

```
LEGO Star Wars: The Complete Saga
Lemmings                          <- different prefix
```

Write `binary_search_title` to return the index of the first match it happens to land on (or `-1`). It does not need to be the first LEGO game in the list.

Check `startswith` first. If the middle title does not match, compare the prefix to the middle title to decide which half to keep searching.

Finding the Other Matches

There may be more matches just before or just after the one you found. The provided function `look_forward_backward(data, start_index, prefix, direction)` collects them.

Its parameters are:

- `data`: the list of games to search
- `start_index`: where to start, which is the index your binary search returned
- `prefix`: the partial title to match, such as "LEGO"
- `direction`: `1` to search forward or `-1` to search backward

The function returns only the matches next to `start_index`, so the game at `start_index` is not included.

Call it twice, once in each direction, then combine the backward matches, the game at `start_index`, and the forward matches into one list:

```
backward_matches = look_forward_backward(data, seed_index, prefix, -1)
forward_matches = look_forward_backward(data, seed_index, prefix, 1)
```

7 Reflection

In a text file named `HW5-reflection.txt`, briefly answer the following:

1. For each of the three functions you wrote, what is its name, what input parameters does it take, and what does it return?
2. What were the biggest problems or challenges you ran into (or was there anything you particularly enjoyed)?
3. About how long did you spend completing the assignment?

When you finish, push your branch and send a pull request to the `hw5` GitHub repository with both `HW5-search.py` and `HW5-reflection.txt`.